

Agentory

Lead-Intelligence Workflow Architecture & Audit

How a single sentence typed by a user becomes qualified companies with cited evidence — every decision, who makes it, and what stops it going wrong.

Field	Value
Date	19 August 2026
Branch	feat/lead-mission-v1
HEAD	48e65c3d
Deployed code	322b4a38 — orchestrate v17, pilot-chat v14, run-agent v21 (all ACTIVE)
Supabase project	ohsdatpvfdjdemstoiuj — one project serves both test and production
Test suite	4,978 passing / 0 failing
Uncommitted	continuation checkpoint fix + regression tests (not deployed)

Decision owners are colour-coded throughout: **GPT** decides, **CODE** validates and executes, **PROVIDER** returns facts.

1. The governing principle

Agency is GPT-first. That phrase is load-bearing and worth stating precisely, because the architecture was rebuilt around it after a run that spent real money answering a question nobody asked.

GPT understands the job

- > GPT plans the work
 - > code validates and safely executes
 - > actors produce facts
 - > GPT evaluates the actual results
 - > GPT replans if necessary

The division of labour

GPT owns (intelligence)	Code owns (safety & execution)
What the user actually wants	That the actor exists and is registered
What evidence is required	That input matches the live schema
Which actors to use, and in what order	Enums, limits, cost ceilings, deadlines
Whether multiple actors are needed	Containment — no stage the mission did not authorise
What inputs to give them	Deduplication and normalisation
Whether results are sufficient	Idempotency and no duplicate paid work
What information is still missing	State persistence and checkpointing
Whether to broaden, narrow or continue	Never auto-spending on a person
Which model tier a task needs	Refusing, and explaining the refusal

The rule that decides every boundary dispute: if being wrong costs money or misdirects the run, GPT decides and code checks. If being wrong corrupts state or spends without authority, code decides and GPT is told.

2. End-to-end workflow

The complete path, with the owner of each decision. Stages that buy nothing are marked; every paid call is a validated actor invocation.

```

USER REQUEST          "Find 10 qualified AI startups in the US currently hiring"
|
v
[1] PILOT / CHAT          pilot-chat
|   classifies intent, opens a conversation, no spend
|   v
[2] MISSION COMPILATION          GPT  gpt-4.1
|   the sentence -> LeadMissionV1
|   verticals, stages, locations, required_signals,
|   required_evidence, hard_constraints, requested_count
|   NO PROPOSAL => MissionCompilationBlockedError (no regex fallback)
|   v
[3] CAPABILITY GATE          CODE
|   approves/rejects the capabilities GPT asked for
|   v
[4] CAPABILITY GRAPH          CODE
|   containment only: which stages MAY run, in what order,
|   which actors each stage MAY reach
|   routing_advisories carry facts GPT cannot infer
|   v
[5] EXECUTION PLAN          GPT  gpt-4.1
|   the whole chain: capability + actor + purpose + input
|   + which paid stages are UNNECESSARY
|   v
[6] PLAN VALIDATION          CODE
|   actor exists / declared by that capability /
|   not_for / cohort_scope / people-stage refusal /
|   dependencies backwards / <= 8 steps
|   refusal -> fed back -> ONE repair round
|   v
[7] DISCOVERY          GPT + PROVIDER
|   GPT picks actors and writes inputs; code compiles and clamps
|   v
[8] INSPECT          CODE
|   counts only, never a verdict:
|   rows returned / identifiable / carrying required evidence
|   v
[9] REPLAN?          GPT  gpt-4.1
|   shown the facts; may re-ask the SAME actor a better question
|   bounded by maxDiscoveryPasses (default 2)
|   v
[10] DEDUPE          CODE
|   keyed on LinkedIn URL / domain / id, global across actors
|   v
[11] TRIAGE + BUDGET + RANK          GPT  gpt-4.1-mini
|   escalates to gpt-4.1 when the pool nears the quota
|   v
[12] IDENTITY RESOLUTION          PROVIDER
[13] ENRICHMENT          PROVIDER
|   v
[14] QUALIFICATION          GPT  gpt-4.1
|   verdict on cited evidence; citations must exist in the registry
|   v
[15] PERSISTENCE          CODE
|   v
[16] QUOTA CHECK          CODE
|   qualified >= requested ?  YES -> done
|                           NO  -> checkpoint + continue
|   v
[17] CONTINUATION          CODE
|   persist checkpoint -> release lease -> dispatch successor
|   successor restores frontier and resumes the SAME plan

```

3. The capability graph

The containment boundary. A capability defines which actors may be reached and what the stage produces; it does *not* rank them. Widening a provider list does not make a bad choice likelier — the validator and the playbook still decide what may run.

Capability	Providers it may reach	Owner of the choice
startup_company_discovery	memo23 · solidcode · linkedin_company_search	GPT
general_company_discovery	memo23 · solidcode · linkedin_company_search	GPT
company_identity_resolution	linkedin_company_search	code (single provider)
company_enrichment	linkedin_company_details	code (single provider)
hiring_verification	linkedin_job_search	GPT may SKIP it
company_brain_qualification	— (no actor)	GPT judges
persistence	— (no actor)	code
founder_discovery	company_employees · people_search	OFFER ONLY — never planned
contact_enrichment	people_search	OFFER ONLY — never planned
job_discovery	4 job boards — uncarded	undriveable; no verified schema

People stages are refused outright by three independent guards. The mission compiler strips them, the graph refuses to insert them, and `validateExecutionPlan` blocks them with their own reason. A run produces qualified companies plus an unlock the user presses; it never buys a person on its own initiative.

4. The actor playbook

Every actor carries a card read from the **live build schema** (`GET /v2/acts/{id}/builds/default`), not from documentation. A 2026-08-01 benchmark found seven places where docs and runtime disagreed; runtime won each time.

What GPT is shown, per actor

Field	Purpose
best_for / not_for	Verified operational knowledge. not_for is enforced , not advisory.
cohort_scope	A fixed population the actor cannot see past (e.g. the YC directory).
input_contract.fields	Every field with its real type , enum and default.
input_contract.example	A minimal known-good input. Copy the shape, not the values.
known_defects	Observed runtime failures with mitigations.
quality	Store traffic, success rate, rating — maturity as a planning signal.
cost_model	Tier and per-result price.

Registered actors and their measured maturity

Actor	Role	Runs	30-day users	Signal
harvestapi/linkedin-company	enrichment	9,299,972	3,245	mature
harvestapi/linkedin-profile-search	people (offer)	7,090,553	5,859	mature
harvestapi/linkedin-company-employees	people (offer)	4,322,317	4,526	mature
harvestapi/linkedin-job-search	hiring verify	1,381,328	707	mature
harvestapi/linkedin-company-search	identity / discovery	962,213	1,449	reliable
memo23/y-combinator-scraper	YC cohort discovery	993	35	niche — 4.62/5, 97.7% ok
solidcode/ycombinator-scraper	YC fallback	278	2	very low traffic

The traffic gap is a real planning input. An unexpected empty result from a 993-run actor is more likely a filter problem than proof the cohort is empty; the same result from a 9.3M-run actor is more likely the truth.

5. Model routing

One policy table decides which model every stage runs on. Tiers used to be hardcoded at each call site, which meant they were *constants* and could not see the run they belonged to. Escalation is about stakes, not workload.

Stage	Tier	Why
mission_compilation	reasoning	one call decides what the whole run is for
discovery_actor_selection	reasoning	chooses which paid actors run
discovery_actor_selection_repair	reasoning	a repair is harder than the attempt that failed
execution_plan	reasoning	the most consequential single call
execution_plan_repair	reasoning	a refused plan is harder to fix than to make
execution_plan_amendment	reasoning	re-plans against the real pool
mission_evaluation	reasoning	the verdict is the product
company_qualification	reasoning	what the user acts on
pool_ranking	reasoning	decides what the user sees first
mission_triage	fast	batch classification; wrong => order, not exclusion
semantic_classification	fast	corrected by the evaluator downstream

```
pool 100, requested 10 -> fast      (gpt-4.1-mini)
pool 4,   requested 10 -> reasoning (escalated)
    "the pool is within twice the requested count, so a triage
    verdict decides a lead rather than its position"
```

There is **no downgrade path**. A reasoning stage cannot be cheapened by volume. Every decision is recorded in `ModelRoutingLedger` with stage, tier, model, reason and escalation count.

6. Actor selection: the backend process in full

This is the mechanism the whole architecture exists to protect. It is worth reading closely, because the difference between “GPT picks a tool” and what actually happens here is the difference between a system that can be trusted with a budget and one that cannot.

6.1 Two planners, one precedence rule

Actor selection happens in **two** places, and which one wins is decided, not accidental.

Planner	Question it answers	Scope
gptExecutionPlanner	What is the whole chain, and which paid stages are unnecessary?	every capability
gptDiscoveryPlanner	Which actors discover the pool, and what is each asked?	discovery only

The chain wins. If the execution plan already named actors for a discovery capability, the engine uses them and does *not* ask the discovery planner again — a second, narrower opinion on a decision already made is the duplication this architecture keeps deleting, and the narrower one cannot see the chain it would invalidate. The chain’s actors still pass through the same validator: nothing is trusted because it arrived by a longer route.

```
const plannedHere = plannedActorsFor(executionPlan, cap);
const strategy = plannedHere.length > 0
  ? validateDiscoveryStrategy(plannedHere.map(...), mission, limits) // chain
  : await resolveDiscoveryStrategy();                               // fallback
```

6.2 What GPT is given before it chooses

The payload is assembled from the live catalog and the compiled mission. GPT cannot reach outside it: an actor key absent from `available_actors` names nothing the system can call.

Section	Contents
Agentory briefing	What Agentory is, the full pipeline, and that no later stage repairs an earlier one.
Company Brain	The standing ICP, with precedence stated: the typed request always wins.
<code>available_actors</code>	Per actor: <code>best_for</code> , <code>not_for</code> , outputs, cost tier, confidence, <code>known_defects</code> , <code>cohort_scope</code> .
<code>input_contract</code>	Every field with its real type , enum and default, plus a worked example.
<code>quality</code>	Store traffic, success rate and rating — maturity as a planning signal.
<code>signal_coverage</code>	Which actors can serve each required signal, and the minimum evidence each yields.
<code>unserveable_scenarios</code>	Requests no registered actor can serve, with the verified reason.
<code>execution_advisories</code>	Facts the router knows and the model cannot infer.
<code>limits</code>	<code>max_actors</code> , <code>max_items_per_actor</code> , <code>requested_lead_count</code> .

6.3 The validation pipeline, gate by gate

Every proposed selection runs this gauntlet **in order**. A **block** discards the selection; a **repair** keeps a corrected version and records that it was corrected.

#	Gate	Refusal code	Severity
1	Is the proposal a list at all?	proposal_not_a_list	block
2	Is each selection an object?	selection_not_an_object	block
3	Is the actor in the closed catalog?	unknown_actor	block
4	Is it registered for company_discovery?	actor_not_for_discovery	block
5	Concept mission vs a name matcher?	actor_not_for_semantic_discovery	block
6	Cohort source outside its cohort?	actor_outside_mission_cohort	block
7	Named twice?	duplicate_actor	repair
8	Missing / invalid role?	role_defaulted	repair
9	Filter the live schema lacks?	filter_dropped	repair
10	No primary named?	primary_promoted	repair
11	Over the actor ceiling?	actor_ceiling	repair (trims breadth)
12	Did anything survive?	no_valid_selection	block

Gates 5 and 6 are the two that carry judgement, and both read a *declared property of the actor* rather than a hardcoded rule. Gate 5 fires when the mission names no companies and the actor's card says `not_for: semantic/concept search`. Gate 6 fires when the actor declares a `cohort_scope` the mission does not target — a YC scraper pointed at German integrators does not return worse results, it returns the wrong population.

6.4 Refusal is not the end — it is feedback

A blocked plan used to end discovery outright: nothing selected, nothing discovered, zero leads, and the only record a violation code nobody acted on. Now the refusal is handed back in the model's own feedback channel, verbatim, and it chooses again against the same closed catalog.

```
proposal -> validate -> BLOCKED
|
+-- violations rendered as instructions
|   "apify_linkedin_company_search declares not_for
|   'semantic/concept search' -- this mission discovers by
|   concept (AI) and names no specific companies, so a name
|   matcher cannot produce this cohort"
|
v
SECOND PROPOSAL -> validate -> runs, or blocks honestly

EXACTLY ONE REPAIR ROUND. A second failure means the catalog cannot serve
this request, and saying so is the correct answer.
```


6.5 From a chosen actor to a paid call

Selection is only half of it. The input GPT wrote is compiled, clamped and re-checked before any money moves.

```
GPT proposes { actor_key, role, input_json, rationale }
|
v parsePlannedInput()      JSON string -> object
|                          (strict mode cannot express an open object)
v validateDiscoveryStrategy() 12 gates above
|
v compile<Actor>Input()      field whitelist, enum check, scalar->list
|                          normalisation, count clamped to the
|                          actor's published limit
v inputFingerprint()        stable digest -> "already asked?"
|
v guardedInvoker(plan, invoke, capabilityId)
|  CapabilityContainmentError if this capability may not
|  reach this actor -- the LAST line, at call time
v shouldSkipProviderCall() resume ledger: already bought?
|
v deadline + checkpoint reserve check
|
v APIFY
```

Containment is enforced twice, deliberately. Once at plan time, where a refusal can be explained to GPT and repaired; and once at call time in `guardedInvoker`, where it throws. A plan-time check alone would be a convention; the call-time throw is what makes an off-graph actor unrepresentable rather than merely discouraged.

6.6 After the call: how a bad choice is caught

Every attempt outcome becomes information the planner can act on. This is the rule that generalises — it names no actor and no field, so it applies to every actor and every request.

Outcome	What it means	What GPT is told
ok	Rows returned	—
empty	The actor worked; the question was too narrow	“widen or drop the most restrictive filters ... do not repeat the identical input”
error	The actor <i>rejected</i> the input	“contract failure rather than an empty result — no retry of the same input can succeed”
skipped_not_configured	Unavailable in this environment	“treat it as unavailable for this run”
compile_failed	Never reached a provider	validator violation (as above)

And “already tried” means **actor + input, not actor**. Keying on the actor made the single most useful recovery unreachable: the same actor, asked a better question. The fingerprint is taken *after* validation, because the validator normalises what it keeps — comparing a raw proposal to a validated selection never matches, and the guard would silently pass everything.

7. Safety: what code refuses, and how it explains itself

The validator is a guardrail, not a strategist. It never picks an actor. It refuses an impossible one and hands the reason back so GPT can choose again — exactly one repair round, because a second failure means the catalog cannot serve the request, and saying so is the honest answer.

Refusal	Meaning
actor_not_for_semantic_discovery	A name matcher asked to discover a concept cohort.

Refusal	Meaning
actor_outside_mission_cohort	A cohort source pointed outside its fixed population.
actor_not_declared_by_capability	The capability may not reach that actor.
capability_not_in_mission_plan	The mission never authorised that stage.
people_stage_never_automatic	Spending on a person is offered, never planned.
actor_returned_no_rows	The actor worked; the question was too narrow.
actor_rejected_input	Contract failure — no retry of that input can succeed.
no_capable_discovery_actor	Nothing registered can serve this cohort. Run stops, \$0 spent.

The generalising rule

Every failed attempt becomes information the planner can act on — and **“already tried” means actor + input, not actor**. That second half matters: keying on the actor made the single most useful recovery (the same actor, asked a better question) structurally unreachable.

```
memo23 industries: ["Engineering, Product and Design"] -> 0 rows
memo23 (no industries)                                -> 20 rows
               ^ previously impossible
```

8. Continuation and state

A request is bigger than one ~125-second invocation. The run slices itself, checkpoints, and resumes — carrying GPT's plan forward unchanged rather than re-deciding it.

```
quota unmet, frontier remains
-> build resume checkpoint (per-company stage state)
-> PERSIST checkpoint          <-- durable BEFORE anything else
-> release the claim lease
-> dispatch successor          <-- fire-and-forget, safe now
-> parent performs NO further write
-> successor MERGES its early write over the inheritance
-> successor restores frontier
-> continuation resumes the same plan
```

Guarantees

Guarantee	Mechanism
Successor cannot outrun its checkpoint	Checkpoint persisted before dispatch.
Parent cannot overwrite successor	Only a <i>failed</i> handoff writes again.
Successor cannot erase its inheritance	Early write merges; never replaces. (fix pending deploy)
No duplicate consumption	Claim lease via <code>claim_sourcing_continuation</code> (SELECT FOR UPDATE).
Retry after failed dispatch	Checkpoint survives; <code>stopped_reason: dispatch_failed</code> .
Empty restore never destroys state	Guard refuses when <code>expected > 0</code> and <code>restored = 0</code> .

9. Audit: what changed, and what it cost to learn

Six defects were found and fixed. **Four of them were invisible to 4,900 unit tests**, because every test injects the model or the provider. They were found only by running the system against the real APIs.

#	Defect	Consequence	Found by
1	Router discarded an <i>approved</i> capability on a membership test	"AI startups" answered by a name matcher: 50 newsletters, 0 qualified	forensic audit
2	GPT actor selection wired to one capability branch only	Planner, catalog, <code>not_for</code> and briefing all bypassed	forensic audit
3	Both planner schemas sent <code>additionalProperties: true</code>	OpenAI strict mode refused 100% of planner calls , silently	live run
4	<code>required</code> not updated after a field rename	Same silent 100% failure, one run later	live run
5	Scalar where a list belongs, iterated character by character	31 bogus violations ("E", "n", "g"...), capability killed	live run
6	Successor's early write replaced the parent's result	Checkpoint destroyed 4s before restore; 51 candidates stranded	live run

The lesson worth keeping. Defects 3–6 all share a shape: a boundary the tests could not cross — the real OpenAI schema validator, the real Apify input parser, a real second invocation. A test that injects the model cannot see a prompt the model would reject. Structural checks now cover 3 and 4 (`strictSchemaContract.test.ts`) without needing a network call.

10. Verified behaviour

Live end-to-end — real OpenAI, real Apify, real spend

REQUEST	"Find 2 qualified AI/startup companies in the US currently hiring"		
MISSION	gpt_validated; verticals [AI, startup]; requested 2		
PLAN	memo23 -> company_search -> company_details -> qualify -> persist hiring_verification NOT planned: memo23 carries openJobs		
ACTOR 1	memo23	[LIVE 23.5s]	-> 20 rows
ACTORS	linkedin_company_search	[LIVE ~8s x10]	-> 38 rows
ACTOR 12	linkedin_company_details	[LIVE 11.1s]	-> 4 rows
QUALIFY	flocksafety.com	QUALIFIED pass	98
	deepgram.com	QUALIFIED pass	98
	checkr.com	insufficient_evidence	
	shipbob.com	insufficient_evidence	
RESULT	2 of 2 QUALIFIED	12 actor calls	capability_plan_complete

Discovery, before and after

Metric	Run 25f3ff57 (18 Aug)	Run 85192217 (19 Aug)
Companies discovered	100 (7% relevant)	100
Open roles evaluated	0	314
Discovery input	searchQuery: "AI" / "startup"	memo23 cohort, correctly typed
Pool composition	26 accelerators, 22 newsletters, 1 podcast	YC companies with embedded jobs
Qualified	0	1 (run stopped early — see §11)

11. Open items

Item	Status	Impact
Continuation checkpoint fix	fixed, not deployed	Runs stop after round 1; frontier stranded. Needs a watched live run.
agentry_trace not persisted	open	Failures must be reconstructed from edge logs. toRouteResultShape drops it.
Continuation harness tests	open	Persistence-failure, dispatch-failure and idempotency paths untested; no HTTP test double exists.
Concept cohorts return zero	by design	Manufacturers/agencies refuse with a stated reason until a cohort actor is carded.
Founder/contact is offer-only	by design	"...then find the founder" yields companies + an unlock button, not contacts.
Qualification precision	watch	A Manufacturing Engineer role passed an "AI startups hiring" mission. Re-check at volume.
provenance mislabel	known defect	requested_count provenance reads explicit_user_request for a value that did not survive.

Recommended next live test

“Find 10 qualified AI startups in the US currently hiring” — after deploying the continuation fix. Success is `continuation_restored` with ~100 records rather than `continuation-restore-empty`, and the run working through the 51-candidate frontier across continuations instead of parking at ready.

Assessment: the architecture is now **GPT plans** → **code safely executes** → **GPT observes** → **GPT replans**. No code path chooses an actor or writes an actor's input. The remaining deterministic surfaces are containment, validation, budget and state — which is where they belong.